

PRÁCTICA DIRIGIDA: Metaheurísticas y Optimización Global

Curso: Inteligencia Artificial (7mo Ciclo - Ciencia de la Computación)

Objetivo: Comprender la mecánica interna, fortalezas y patologías de los algoritmos de búsqueda local y poblacional mediante ejecución manual algorítmica e implementación computacional pura.

PARTE 1: RESOLUCIÓN A LÁPIZ Y PAPEL (EJECUCIÓN MANUAL)

Instrucciones: En esta sección, usted actuará como la CPU. Deberá calcular y registrar el estado de las variables en cada iteración. Use calculadora para las funciones trascendentales y probabilidades. Redondee a 3 decimales.

MÓDULO 1: HILL CLIMBING (Ascenso de Colina)

Ejercicio 1.1: Convergencia Exitosa (Función Convexa Local)

Enunciado: Optimice la función unidimensional $f(x) = -(x - 3)^2 + 10$. El objetivo es **Maximizar**.

- **Estado inicial:** $x_0 = 1,0$
- **Generador de vecinos:** $N(x) = \{x - \Delta, x + \Delta\}$ con paso $\Delta = 1,0$.

Desarrollo Guiado:

Complete la siguiente tabla de iteraciones. El algoritmo se detiene cuando ningún vecino mejora el estado actual estrictamente.

Iter	Estado Actual (x_t)	$f(x_t)$	Vecino Izq ($x - \Delta$)	$f(x - \Delta)$	Vecino Der ($x + \Delta$)	$f(x + \Delta)$	Decisión (Mejor Vecino)
0	1.0						
1							
2							
3							

Pregunta de análisis: ¿En qué iteración se detiene el algoritmo? ¿Coincide el estado final con el óptimo global de la parábola?

Ejercicio 1.2: El Colapso en la Trampa Local

Enunciado: Considere el problema de las 4-Reinas. El objetivo es **Minimizar** el número de ataques entre reinas.

- **Representación del estado:** Vector de 4 enteros $V = [c_1, c_2, c_3, c_4]$, donde c_i es la fila de la reina en la columna i .
- **Función objetivo $h(V)$:** Número de pares de reinas atacándose directa o diagonalmente. (Meta $h = 0$).
- **Generador de vecinos:** Mover una sola reina a una celda diferente en su misma columna. (Existen $4 \times 3 = 12$ vecinos por estado).
- **Estado inicial:** $V_0 = [2, 4, 1, 3]$ (Asumiendo índice 1 a 4). Evalúe si se atacan.

- **Estado propuesto:** Asuma que el estado actual es $V_{trampa} = [1, 2, 3, 4]$ (Todas en la diagonal principal).

Desarrollo Guiado:

1. Calcule $h(V_{trampa})$.
2. Genere 3 vecinos posibles moviendo la reina de la columna 1. Calcule la heurística para esos 3 vecinos.
3. *Explicación:* Observe los valores heurísticos generados. ¿Logra alguno de estos movimientos simples reducir la heurística a un valor menor que $h(V_{trampa})$? Si la respuesta es no para todos los 12 vecinos, ¿qué le ocurre al algoritmo Hill Climbing puro en este punto?

Ejercicio 1.3: La Meseta (Plateau)

Enunciado: Minimizar $f(x) = |x|$ si $|x| > 2$, y $f(x) = 2$ si $|x| \leq 2$.

- **Estado Inicial:** $x_0 = 1, 0$.
- **Generador:** $N(x) = \{x - 0,5, x + 0,5\}$.

Desarrollo Guiado:

Muestre la evaluación de la iteración 0. Explique formalmente por qué el algoritmo estándar falla instantáneamente, a pesar de no estar en el óptimo global ($x = 0, f(x) = 2$). ¿Qué variante de Hill Climbing solucionaría este problema específico y qué riesgo introduce dicha variante?

MÓDULO 2: SIMULATED ANNEALING (Recocido Simulado)

Ejercicio 2.1: El Escape Probabilístico

Enunciado: Estamos en un problema de **Minimización**. El sistema intenta escapar de un mínimo local profundo.

- **Estado actual:** S_t con $E(S_t) = 50$.
- **Vecino generado:** S_{new} con $E(S_{new}) = 65$. (Observe que es un empeoramiento de $\Delta E = +15$).
- **Criterio de Metrópolis:** $P(\text{aceptar}) = e^{-\frac{\Delta E}{T}}$.

Desarrollo Guiado:

Suponga que dispone de un dado de 100 caras (generador uniforme $U(0,1)$). En la iteración actual, el dado arroja el valor **0.35**. Calcule la probabilidad de aceptación y determine si el estado S_{new} es aceptado para las siguientes tres temperaturas:

Temperatura (T)	ΔE	$P(\text{aceptar}) = e^{-\frac{\Delta E}{T}}$	Dado Aleatorio	Decisión (Aceptar/Rechazar)
100	+15		0.35	
20	+15		0.35	
2	+15		0.35	

Pregunta de análisis: Analice la columna de decisiones. ¿Cómo demuestra este ejercicio el principio de “exploración inicial, explotación final” del Recocido Simulado?

Ejercicio 2.2: Rastreo del Enfriamiento

Enunciado: Maximizar $f(x) = x \sin(x)$.

- **Estado inicial:** $x_0 = 8,0$.
- **Generador:** $x_{new} = x_{actual} + U(-1, 1)$.
- **Temperatura Inicial:** $T_0 = 5,0$.
- **Factor de enfriamiento:** $\alpha = 0,5$ (donde $T_{k+1} = \alpha \cdot T_k$).
- (Dado simulado) Para Δx : $\{+0,5, -0,8\}$. Para Metrópolis $U(0, 1)$: $\{0,45, 0,12\}$.

Desarrollo Guiado:

Nota: Como es maximización, $\Delta E = f(S_{actual}) - f(S_{new})$. Un empeoramiento es una reducción del valor de la función, generando un $\Delta E > 0$.

Iter (k)	T_k	x_k	$f(x_k)$	Δx	x_{new}	$f(x_{new})$	ΔE	P(aceptar)	U(0,1)	Decisión
0	5.0	8.0		+0.5	8.5				0.45	
1		Dep.		-0.8	Dep.				0.12	

Nota: “Dep.” significa que el valor depende de la decisión de la iteración anterior.

Ejercicio 2.3: La Paradoja de la Extinción Prematura (Quenching)

Enunciado: Un desarrollador configura el enfriamiento como $T_{k+1} = T_k - 100$, con $T_0 = 250$. En 3 iteraciones, la temperatura es negativa o cero.

Desarrollo Guiado: Explique matemáticamente qué ocurre con la ecuación de probabilidad $e^{-\frac{\Delta E}{T}}$ cuando $T \leq 0$. ¿En qué algoritmo clásico se convierte exactamente el Recocido Simulado bajo esta mala parametrización?

MÓDULO 3: ALGORITMOS GENÉTICOS

Ejercicio 3.1: Operadores Evolutivos Discretos (Mochila Binaria)

Enunciado: El Problema de la Mochila (Knapsack). Tenemos 5 objetos con pesos $W = [2, 3, 4, 5, 9]$ y valores $V = [3, 4, 8, 8, 10]$. Capacidad máxima $C = 15$.

- **Individuo:** Cromosoma binario $[b_1, b_2, b_3, b_4, b_5]$.
- **Fitness $F(x)$:** Suma de valores si peso total $\leq C$. Si peso $> C$, $F(x) = 0$ (muerte).

Desarrollo Guiado (Simulación de 1 Generación):

1. Evaluación de Población Inicial:

- $I_1 = [1, 0, 1, 0, 1] \rightarrow$ Peso: _____ Valor (Fitness): _____
- $I_2 = [0, 1, 1, 1, 0] \rightarrow$ Peso: _____ Valor (Fitness): _____
- $I_3 = [1, 1, 1, 1, 0] \rightarrow$ Peso: _____ Valor (Fitness): _____
- $I_4 = [0, 0, 0, 1, 1] \rightarrow$ Peso: _____ Valor (Fitness): _____

2. Selección por Ruleta:

Basado en sus valores de Fitness, calcule la probabilidad de selección $P_i = \frac{F_i}{\sum F_j}$ para cada individuo. (Observe que los individuos infractores tienen 0 %).

3. Cruce (Crossover de 1 punto):

Asuma que I_1 e I_2 son seleccionados. El punto de corte es después del gen 3. Muestre los cromosomas de los dos Hijos (H_1, H_2).

4. Mutación (Bit-flip):

El gen 5 de H_1 sufre mutación. Muestre el cromosoma final de H_1 y calcule su nuevo Fitness. ¿Mejoró o empeoró respecto a sus padres?

Ejercicio 3.2: Cruce Aritmético Continuo

Enunciado: Optimización en $\mathbb{R}^2$.

- Padre 1: $P_1 = (10, 0, 5, 0)$
- Padre 2: $P_2 = (0, 0, 15, 0)$
- Operador: Cruce aritmético con factor $\beta = 0,3$.
 $H_1 = \beta P_1 + (1 - \beta)P_2$
 $H_2 = (1 - \beta)P_1 + \beta P_2$

Desarrollo Guiado:

Calcule las coordenadas exactas de los hijos H_1 y H_2 . Si representamos los padres como puntos en un plano cartesiano, ¿dónde se ubican geoméricamente los hijos respecto a los padres?

Ejercicio 3.3: La Pérdida de Diversidad (Análisis Crítico)

Enunciado: En un problema TSP (Agente Viajero), después de 50 generaciones, los 100 individuos de la población son permutaciones exactamente idénticas.

Desarrollo Guiado:

¿Cómo se denomina este fenómeno en computación evolutiva? Identifique dos parámetros del Algoritmo Genético que el programador probablemente configuró mal (uno muy alto, y uno muy bajo) para causar esta catástrofe.

MÓDULO 4: ANÁLISIS COMPARATIVO

Responda fundamentadamente:

1. Si la función a optimizar es estrictamente convexa (forma de tazón perfecto sin arrugas), ¿cuál de los tres algoritmos usaría y por qué?
2. Usted tiene un problema donde evaluar la función objetivo $f(x)$ toma 5 minutos de procesamiento (ej. simulación de fluidos). ¿Usaría un Algoritmo Genético con población de 1000 individuos? Proponga la alternativa más lógica justificando los recursos computacionales.

PARTE 2: IMPLEMENTACIÓN EN PYTHON (DESDE CERO)

Instrucciones: Abra un entorno local (VS Code/Jupyter) o Google Colab. Usted deberá programar la lógica base de las tres metaheurísticas. Está prohibido el uso de librerías de optimización (como `scipy.optimize` o frameworks como DEAP). Solo se permite `numpy`, `math`, `random` y `matplotlib`.

1. Definición del Benchmark

Implemente la función de Rastrigin en 2D (típicamente evaluada en $[-5,12,5,12]$). Es un problema de minimización altamente multimodal.

$$f(x, y) = 20 + x^2 - 10 \cos(2\pi x) + y^2 - 10 \cos(2\pi y) \quad (1)$$

(El óptimo global es 0 en (0,0)).

2. Implementación de Algoritmos

A) Hill Climbing:

- Implemente una función `hill_climbing(f, bounds, step_size, max_iter)`
- Retorne la mejor solución encontrada y la lista de valores de la función por iteración (curva de convergencia).

B) Simulated Annealing:

- Implemente `simulated_annealing(f, bounds, step_size, T_init, alpha, max_iter)`
- Asegúrese de registrar cuándo se acepta un empeoramiento.
- Retorne la curva de convergencia.

C) Algoritmo Genético (Representación Real):

- Implemente `genetic_algorithm(f, bounds, pop_size, p_cross, p_mut, num_gens)`
- Use Selección por Torneo (tamaño 3).
- Use Cruce Aritmético Uniforme.
- Use Mutación Gaussiana en las coordenadas.
- Registre el *Mejor Fitness* de cada generación.

3. Taller de Experimentación y Reporte

Laboratorio Empírico:

Ejecute los 3 algoritmos sobre la función de Rastrigin 2D un total de 10 veces cada uno (para aislar la varianza estocástica).

1. **Tuning de Parámetros:** Pruebe al menos 3 tasas de enfriamiento para SA ($\alpha \in \{0,80, 0,95, 0,99\}$). Muestre qué pasa con la convergencia.
2. **Gráfica Comparativa:** Usando `matplotlib`, grafique en una sola figura las tres curvas de convergencia (promedio de las 10 corridas). Eje X: Iteraciones/Generaciones, Eje Y: Valor de la función Rastrigin.
3. **Análisis de Resultados:**

- ¿Cuál algoritmo convergió más rápido?
 - ¿Cuál algoritmo encontró sistemáticamente una solución más cercana al $(0, 0)$?
 - Si SA superó a HC, documente una ejecución donde SA “saltó” un valle local temporalmente aumentando su función objetivo.
-

Entregables y Evidencias de Aprendizaje

El estudiante deberá subir a la plataforma institucional:

1. Documento PDF con las tablas y respuestas de la **PARTE 1** (puede ser manuscrito y escaneado legiblemente, demostrando su capacidad de aplicar algoritmos paso a paso).
2. Archivo `.ipynb` (Notebook) o `.py` con el código fuente estructurado, fuertemente comentado, y las gráficas generadas en la **PARTE 2** (demostrando implementación correcta desde cero).
3. Breve conclusión analítica (1 página) sobre qué metaheurística considera superior para este landscape basándose en la varianza de sus resultados empíricos (demostrando comprensión y análisis crítico).